Captain's UI Library

Native-looking Factorio 2.1 GUI helpers for mods

This guide explains how to install the library, render the decider editor, wire GUI events, evaluate red/green circuit inputs, serialize editor state, and integrate the result into another mod.

Condition editor with AND/OR joiners

What the library gives you

- A native-looking decider editor with Conditions, Outputs, and Else outputs.
- Reusable state helpers for create, normalize, evaluate, move, and serialize.
- Tagged GUI event handlers so consuming mods do not duplicate row logic.
- Wire-aware red/green evaluation for condition inputs and input-count outputs.

1. Quick Start

Install the library and render your first editor

Dependency

Add Captain's UI Library as a dependency in the consuming mod's info.json.

```
{
  "dependencies": [
    "base >= 2.1.0",
    "captainjhory-ui-library"
  ]
}
```

```
local ui = require(
  "__captainjhory-ui-library__.scripts.ui"
)
```

Minimal Render

Keep editor state in storage, normalize it before use, and render it into any parent GUI element.

```
storage.my_editor_state = storage.my_editor_state
    or ui.decider_state.new_state()

local state = ui.decider_state.normalize_state(
    storage.my_editor_state
)

ui.decider_editor.add(parent, state)
```

This renders the editor. Event handling is covered on the next page.

2. Public API Map

The library is split into small modules

Module	Purpose
ui.decider_state	Create, normalize, evaluate, move, and serialize editor state.
ui.decider_editor	Handle tagged GUI events and mutate editor state.
ui.components	Render native-looking GUI components from state.
ui.relative_panel	Open and close GUI panels anchored to Factorio relative GUI targets.
ui.mod_primitives	Low-level helpers used by higher-level components.

Recommended Ownership

- Your mod owns persistent state in storage.
- The library owns GUI shape, tags, rendering, and event mutation helpers.
- Your mod owns domain behavior: reactor logic, real signal values, and how serialized outputs affect entities.

3. Wire GUI Events

Let the helper mutate state and rebuild only when needed

Pattern

```
local function rebuild(player_index)
  local player = game.get_player(player_index)
  if not player then return end

  -- Open or rebuild your panel here.
  ui.decider_editor.add(parent, get_state())
end

local function handle_editor_event(event, handler)
  if handler(get_state(), event) then
    rebuild(event.player_index)
  end
end
```

```
script.on_event(defines.events.on_gui_click, function(event)
  handle_editor_event(event, ui.decider_editor.handle_click)
end)
```

Handlers

- handle_click
- handle_checked_state_changed
- handle_selection_state_changed
- handle_text_changed
- handle_gui_elem_changed

Each handler returns true only when the visible editor should rebuild. This keeps immediate UI feedback while avoiding needless rebuilds for edits that do not affect visual state.

4. State Model

The editor state is plain Lua data

Create And Normalize

```
local state = ui.decider_state.new_state()
state = ui.decider_state.normalize_state(state)
```

Normalization fills missing defaults and protects old saves when the library gains new fields.

Field	Meaning
conditions	Condition rows: signals, comparator, joiner, and wire toggles.
outputs	Rows used when the condition group is true.
else_outputs	Rows used when the condition group is false.
conditions_fulfilled	Runtime visual/evaluation flag.

Move Rows

```
ui.decider_state.move_row(
  state.conditions,
  from_index,
  to_index
)
```

The UI exposes tiny movement buttons because native decider row dragging is engine behavior, not a general mod GUI API. The helper keeps row movement predictable.

Joiners and move controls

5. Wire-Aware Evaluation

Red and green checkboxes filter signal input

Signal Input Shape

```
ui.decider_state.evaluate_decider_editor(state, {
  red = {
    ["virtual/signal-T/normal"] = 500,
  },
  green = {
    ["item/uranium-fuel-cell/normal"] = 2,
  },
})
```

When red and green tables are present, condition input checkboxes decide which tables are read. If both are enabled, values are added together, matching Factorio circuit behavior.

Runtime Results

- condition.fulfilled is set for each condition row.
- state.conditions_fulfilled is true when the AND/OR group evaluates true.
- Normal outputs are fulfilled when conditions are true.
- Else outputs are fulfilled when conditions are false.
- Input-count outputs store output.resolved_count.

Active output row

6. Serialization

Convert editor state into domain data

Serialize

```
local serialized = ui.decider_state
  .serialize_decider_editor(state)

-- Save or transform serialized data into
-- your mod's domain-specific config.
```

Serialization includes condition input wires, condition output wires, output input wires, joiners, comparator, selected signals, and constants. Runtime-only visual fields are not intended to be your domain logic.

Important Boundary

- The library tells you what the user configured.
- Your mod decides how that config affects entities, requests, recipes, or circuit behavior.
- Output wire checkboxes are serialized for the consuming mod to use when applying output behavior.

7. Relative Panels

Anchor your editor beside native Factorio GUI

Open

```
local options = {
  name = "my_mod_panel",
  caption = {"my-mod.panel-title"},
  direction = "vertical",
  anchor = {
    gui = defines.relative_gui_type.controller_gui,
    position = defines.relative_gui_position.right,
  },
}

ui.relative_panel.open_relative_panel(
  player,
  options,
  build_content
)
```

Close

```
ui.relative_panel.close_relative_panel(
  player,
  options
)
```

The helper destroys the previous panel before opening a fresh one, which makes rebuilds simple and avoids stale GUI elements.

Condition row inside a relative panel

8. Production Checklist

Before using the library in a real mod

Checklist

- Store editor state under your mod's own storage table, not inside the library.
- Normalize state after migrations and before rendering.
- Pass real red/green signal tables when evaluating fulfilled state.
- Rebuild the visible panel after handlers return true.
- Serialize editor state before saving domain config or applying logic.
- Use output_wires and input_wires in your own domain logic where they matter.
- Keep the library demo setting disabled in normal play.

Next recommended step for Smart Reactors: replace the current reactor-specific editor with ui.decider_editor, then translate serialized outputs into fuel request behavior.